ADR-001: 采用分层架构+MVC模式的模块化单体架构

状态

已采纳

背景

项目概况

本项目为"校园二手交易平台",主要面向校内学生提供二手商品发布、浏览、搜索、收藏、沟通、订单管理、评价等功能。

项目约束

- **项目周期**: 10周
- **团队规模**: 4人
- **技术背景**: 团队成员对前后端开发有基础,但没有完整微服务开发经验
- **项目目标**: 完成可运行原型和架构文档,而不是支撑真实大规模并发

需要解决的问题

如何选择合适的架构风格,以平衡以下需求:

1. 在10周内完成可运行系统
2. 保证代码可维护性和可扩展性
3. 控制团队学习成本
4. 为未来可能的扩展预留空间

决策

本项目采用**前后端分离** + **分层架构** + **MVC模式的模块化单体架构**。

架构组成

1. **前端层**: 负责页面展示、用户交互和接口调用
2. **后端分层架构**:
    - **Controller层**: 接收请求、参数校验、返回响应
    - **Service层**: 处理业务逻辑
    - **Repository/Mapper层**: 访问数据库
3. **数据库层**: 统一存储在关系型数据库中

模块划分

后端内部按业务划分为以下模块:

- 用户模块(User Module)
- 商品模块(Good Module)
- 订单模块(Order Module)
- 评价模块(Rating Module)
- 消息模块(Comment Module)
- 收藏模块(Favorite Module)
- 举报模块(Report Module)

决策理由

1. **符合项目规模**: 校园二手交易平台属于中小型系统,不需要微服务架构

2. **开发成本可控**: 分层架构能够让团队快速完成核心功能
3. **便于团队分工**: 前后端可以并行开发,后端内部可按模块划分
4. **可维护性较好**: Controller、Service、Repository分层避免业务逻辑堆积
5. **后续仍有扩展空间**: 模块化设计为未来拆分服务保留可能

后果

积极影响

1. **快速交付**: 10周内完成可运行系统的可行性最高
2. **降低复杂度**: 避免微服务带来的服务通信、部署、监控等额外复杂度
3. **团队协作**: 清晰的分层和模块划分便于团队成员并行开发
4. **学习成本低**: 团队成员容易理解和实现MVC架构
5. **调试简单**: 单体应用调试和测试更加直观
6. **部署方便**: 只需部署一个后端应用,降低运维成本

消极影响

1. **扩展性受限**: 单体架构的整体扩展能力不如微服务
2. **耦合风险**: 如果模块边界不清晰,后期代码可能变得臃肿
3. **局部影响**: 所有功能部署在一个应用中,局部修改可能影响整体系统
4. **技术栈统一**: 难以为不同模块选择最适合的技术栈

风险

1. **"大泥球"风险**: 如果不严格遵守分层和模块边界,单体应用容易变成难以维护的"大泥球"
    - **缓解措施**: 制定代码规范,定期进行代码审查,严格遵守模块边界
2. **业务逻辑分散风险**: 如果业务逻辑写在Controller中,会导致维护困难
    - **缓解措施**: 明确规定Controller只负责请求响应,业务逻辑必须在Service层实现
3. **数据库设计风险**: 如果数据库表设计不合理,后续扩展会受影响
    - **缓解措施**: 在设计阶段充分考虑业务需求和扩展性,进行数据库设计评审

替代方案

方案A: 微服务架构

描述: 将系统拆分为用户服务、商品服务、订单服务、消息服务、评价服务等独立服务

拒绝原因:

1. 复杂度过高,需要处理服务拆分、服务通信、API网关、认证传递、分布式事务等问题
2. 与课程项目目标不匹配,本项目更强调需求分析、架构设计、核心功能实现
3. 团队经验不足,容易在环境配置和服务联调上消耗过多时间
4. 容易过度设计,当前业务规模较小,使用微服务收益不明显

方案B: 事件驱动架构

描述: 通过事件进行模块之间的解耦,例如用户下单后发布"订单创建事件"

拒绝原因:

1. 核心业务以同步CRUD为主,使用传统分层架构更直接
2. 异步事件链路不如同步调用直观,增加调试和测试难度
3. 引入消息队列等中间件成本较高,不利于快速交付
4. 适合作为局部补充,而不是主体架构

部分采纳: 在订单创建、消息通知等功能中借鉴事件驱动思想,但不引入复杂的消息队列中间件

实施计划

阶段1: 架构搭建(第1-2周)

1. 搭建前后端项目基础结构
2. 实现Controller、Service、Repository分层
3. 配置数据库连接和ORM框架
4. 制定代码规范和模块边界约定

阶段2: 核心模块开发(第3-7周)

1. 按模块优先级依次实现用户、商品、订单等核心功能
2. 严格遵守分层原则,避免跨层调用
3. 保持模块间清晰边界,通过Service接口通信

阶段3: 集成测试与优化(第8-9周)

1. 进行模块集成测试
2. 检查代码质量,重构不合理的部分
3. 性能测试和优化

阶段4: 文档与交付(第10周)

1. 完善架构文档
2. 编写部署文档
3. 准备演示材料

演进路线

虽然当前选择单体架构,但为未来可能的演进预留空间:

1. **短期(0-6个月)**: 保持模块化单体架构,完善核心功能
2. **中期(6-12个月)**: 如果用户量增长,可考虑引入缓存、读写分离等优化
3. **长期(12个月+)**: 如果业务规模持续增长,可将高耦合度较低的模块(如消息通知、搜索推荐)拆分为独立服务

参考

- 架构候选方案对比表
- 架构辩论赛记录
- "Software Architecture Patterns" by Mark Richards
- "Building Evolutionary Architectures" by Neal Ford et al.

决策历史

- 2026-04-30: 团队讨论并初步确定采用分层架构+MVC
- 2026-04-30: 通过架构辩论赛验证决策合理性
- 2026-04-30: 正式采纳此架构方案